

Atlas Architecture Overview

Owner: Architecture Review group · **Last updated:** 10 June 2026 · **Audience:** engineers joining Northwind Labs and anyone changing cross-service behaviour

1. What Atlas does

Atlas is Northwind Labs' data observability platform. Customers connect their data warehouses, pipelines and BI tools; Atlas collects metadata and events, detects anomalies (freshness, volume, schema and distribution changes) and notifies the right people. Beacon, the alerting and on-call add-on, routes those notifications and manages escalation for customers.

2. High-level data flow

1. The **Atlas Collector**, a lightweight agent written in **Go**, runs in the customer's environment (or as a managed connector in our cloud) and gathers query logs, table statistics and pipeline run events.
2. The Collector sends events over HTTPS to the **Ingest Gateway**, which authenticates the API key, validates size and schema, and publishes to Kafka (see RFC-0042).
3. The **ClickHouse sink** consumes Kafka and writes events into ClickHouse, the analytical store for all time-series and event data.
4. The **Monitor Engine** evaluates customer monitors every minute against ClickHouse and creates incidents when thresholds or anomaly models fire.
5. Incidents are passed to the **Beacon Notification Service**, which deduplicates alerts and delivers them to Slack, PagerDuty, email, SMS or webhooks.
6. Customers explore data and configure monitors in the **Atlas Web App**, which talks to the **Query API**.

3. Platform

- All services run on Kubernetes (Amazon EKS 1.30) in two primary regions: eu-west-1 for EU customers and us-east-2 for US customers. Standby regions are described in RFC-0051.
- Infrastructure is defined in Terraform in the `infra-live` repository and deployed with Argo CD.
- Service-to-service traffic uses mTLS via the Linkerd service mesh. The public edge is Envoy behind AWS Network Load Balancers.
- Observability of our own systems uses OpenTelemetry, with metrics in Prometheus and dashboards in Grafana.

4. Components and owners

Component	Language / technology	Owning team	Datastore
Atlas Collector	Go	Integrations	none (stateless agent)
Ingest Gateway	Go	Data Platform	Kafka
ClickHouse sink	Go	Data Platform	ClickHouse
Monitor Engine	Python	Detection	ClickHouse, Postgres
Anomaly models	Python (scikit-learn, Prophet)	Detection	S3 model registry
Query API	Python (FastAPI)	API Platform	ClickHouse, Postgres, Redis
Atlas Web App	TypeScript (React)	Frontend	via Query API
Beacon Notification Service	TypeScript (Node.js)	Beacon	Postgres, Redis
Auth Service	Go	Identity	Postgres
Billing Sync	Python	Growth Engineering	Postgres, Salesforce

5. Datastores

- **Postgres** (`pg-atlas-main`): metadata such as workspaces, users, monitors, incidents and integration settings. PostgreSQL 16 with Patroni; failover procedure in RB-DB-07.
- **ClickHouse** (`ch-atlas-prod`): events and metrics; 6 shards with 2 replicas per region.
- **Kafka**: durable event buffer between ingestion and storage; 72-hour retention.
- **Redis**: query cache, web sessions and rate-limit token buckets, in three separate clusters (RB-CACHE-02).
- **S3**: exports, anomaly model artefacts and backups. Postgres backups are taken every 6 hours and kept for 35 days.

6. Multi-tenancy and isolation

- Every workspace has a `workspace_id`. All ClickHouse tables are ordered by `workspace_id` first and every query issued by the Query API is forced through a row policy that filters on the caller's workspace.
- Enterprise customers can opt for a dedicated ClickHouse cluster ("isolated compute"). Dedicated clusters are provisioned by the Data Platform team within 10 business days.
- API keys are scoped to a single workspace and rate limited per key (RFC-0057).

7. Security boundaries

- Customer credentials for warehouses are stored encrypted in AWS Secrets Manager, never in Postgres.
- The Collector only needs read access to warehouse metadata and query history; it never reads table contents unless the customer enables "sample values".
- Production access for engineers is granted just-in-time through Okta and expires after 4 hours.

8. Key architectural decisions

Decision	Reference	Year
Move event storage from Postgres to Kafka and ClickHouse	RFC-0042	2025
Warm-standby failover per region	RFC-0051	2026
Tier-based API rate limiting with token buckets	RFC-0057	2026

9. Where to learn more

- Engineering Onboarding Guide (setting up a local environment)
- Tech Radar (approved and discouraged technologies)
- Incident Response Runbook RB-INC-01